

เลขที่นั่งสอบ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ข้อสอบปลายภาคการศึกษาที่ 2/2557

วันที่ 11 พฤษภาคม 2558

เวลา 09.00 – 12.00 น.

วิชา CPE 334 Operating System

น.ศ. วศ.คอมพิว.เดอ์ปีที่ 3 หลักสูตรภาษาไทย

คำสั่ง

- ข้อสอบมี 2 ส่วน ส่วนละ 50 คะแนน รวมเป็น 100 คะแนน
 - ส่วนที่ 1 มี 3 ข้อใหญ่
 - ส่วนที่ 2 มี 3 ข้อใหญ่
- ให้ทำข้อสอบส่วนที่ 1 ลงในตัวข้อสอบที่เว้นช่องไว้ให้ ส่วนที่ 2 ทำลงในสมุดคำตอบ
- อนุญาตให้ใช้เครื่องคำนวณ รุ่นและยี่ห้อที่มหาวิทยาลัยกำหนดได้
- ไม่อนุญาตให้นำเอกสารใดๆ เข้าห้องสอบ
- เขียนชื่อและ รหัสประจำตัว ลงในกระดาษคำตอบทุกแผ่น (และแผ่นนี้)

(รศ.ดร.ธีรณี อจลากุล)

ผู้ออกข้อสอบส่วนที่ 1

(อ.ราชวิรัช สโรชวิกสิต)

ผู้ออกข้อสอบส่วนที่ 2

ข้อสอบนี้ได้ผ่านการประเมินจากภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

ส่วนที่ 1

1: (25 points) Short answers: write your answers within the provided space

1.1. What is the relationship between threads and processes?

process สามารถแบ่งเป็นหลายๆ thread ได้
 ทำให้สามารถทำ 1 process ได้เร็วขึ้นด้วยการทำ
 หลาย thread ควบคู่กัน

1.2. What is a race condition? Give an example.

process หลาย process มาพบเจอ shared variable หนึ่งอัน

ถ้าการประมวลผลของ 2 thread ที่มาจาก A B ใช้ shared variable เดียวกัน
ก็จะได้ output เดียวกัน deterministic result ที่ไม่แน่นอน

1.3. What are the requirements of any solution to the critical sections problem? Why are the requirements needed? (hints: mention at least two requirements)

- ข้อที่ต้องมีใน critical section
- mutual exclusion ← ถ้าใครเข้า critical section แล้วคนอื่นเข้าไม่ได้
 - performance
↳ ไม่เปลือง CPU ต้องมี spin-waiting
 - fairness
↳ ไม่ทำให้ใคร starvation

1.4. What is a test-and-set instruction? How can it be used to implement mutual exclusion?

- เป็น atomic instruction
- จะ return ค่าของตัวแปร และ set ค่าใหม่ในตัวแปร
- ใช้ทำ loop spin-wait

while (TestAndSet(mutex → flag, 1) == 1)

while (TestAndSet(flag → 1) == 1)

ชื่อ รหัสประจำตัว ภาควิชา/ชั้นปี

1.5. What is the producer-consumer problem? Give an example of its occurrence in operating systems.

producer จะทำหน้าที่ใส่ค่าใน buffer

consumer จะทำหน้าที่เอาค่าใน buffer ออกมา

ปัญหาคือ ต้องไม่ให้ producer ใส่ค่าถ้า buffer เต็ม

และ consumer เอาค่าออกมาจาก buffer เปล่า

1.6. What is a semaphore? Describe how to implement a lock using semaphores.

semaphore คือ ตัวแปรหนึ่งที่ใช้กับค่าจำนวนหนึ่ง

- sem-init → set ค่าเริ่มต้นของตัวแปร

- sem-wait → ถ้าตัวแปร < 0 นอน ถ้า 0 ยอมทำงานให้ process ที่ call sleep
แต่ถ้า > 0 จะให้ทำงานได้ และลบค่าตัวแปรไป 1 (lock)

- sem-post → บวกค่าตัวแปรไป 1 แล้วปลุก process ที่นอนอยู่มาทำงาน (unlock)

1.7. What is deadlock? What is starvation? How do they differ from each other?

deadlock = ไม่มีใครได้ทำงานเลย

starvation = คนหนึ่งไม่ได้ทำงานสักที

ต่างกันตรงที่ starvation ยีคนอื่นได้ทำงานแล้ว deadlock ไม่มีเลย

1.8. Give an example (with explanation) of a practical use for a semaphore concept. In your given example, a simple mutex is should not be sufficient and a semaphore is then required.

producer-consumer จะใช้ semaphore ในการ lock
และใช้ mutex ธรรมดาไม่ได้ เพราะ producer กับ consumer
ไม่ได้สื่อสารกัน

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

2: (10 points) From the code fragments below, explain what the data structure, "queue_t", is used for in the context of mutual exclusion. Provide detailed explanation of these code fragments. (The back of this page may be used if more space is needed for your answer)

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0;
    m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, getpid());
        m->guard = 0;
        park();
    }
}

void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
```

```

        unpark(queue_remove(m->q)); // hold lock (for next thread!)
        m->guard = 0;
    }

```

จะมี queue ให้ process ที่มารอทำงาน โดย process ใน queue จะรออยู่
เมื่อ process ที่ทำงานอยู่ทำเสร็จ จะส่ง lock ให้ process ใน queue บังคับ
ปลุก process ที่ได้ lock มาทำงาน
แต่ถ้าไม่มี process ใน queue บังคับ ก็คือ unlock

ชื่อ รหัสประจำตัว ภาคเรียนที่

3: (15 points) Suppose we want to program a Web server; we could easily do so without using concurrency, as follows:

```

while(true)
{
    wait connection
    read from socket and parse url
    look up url contents in cache
    if (not in cache)
        fetch from disk
        put in cache
    send data to client
}

```

Because processing only one client at a time is not scalable, how can we adapt this code to work for a lot of clients? Outline the pseudo-code using producer-consumer concept. Explain how the concept is used in your pseudo-code.

เพิ่ม semaphore โดย init ให้
 server = 1 → server ว่าใน client สามารถ
 client = 0 → client ยังไม่พร้อมต่อ server
 mutex = 1 → lock พร้อมในเอาไปใช้

จนบรรทัด wait connection เพิ่ม

sem_wait(&server); → ให้รอ server ว่าว่าง

sem_wait(&mutex); → lock

ใต้บรรทัด send data to client เพิ่ม

sem_post(&mutex); → unlock

sem_post(&client); → ปลุก client มาทำงานต่อ

ชื่อรหัสประจำตัว.....ภาควิชา/ชั้นปี.....

ส่วนที่ 2

1. จากเนื้อหาที่เกี่ยวกับการจัดการอุปกรณ์เชื่อมต่อภายนอก (IO Device) จงตอบคำถามต่อไปนี้ (25 คะแนน)

1.1. อัลกอริทึมของ Disk Scheduling แบบ Shortest Position Time First (SPTF) มีหลักการทำงานอย่างไร ซึ่งอัลกอริทึมนี้ก่อให้เกิดปัญหา Starvation ได้ จงเสนอวิธีแก้ไขโดยละเอียด (10 คะแนน)

ดูจากอนั้นหรืออ่านอยู่ position ไหน แล้วไปอ่านข้อมูล
โดยจะไล่ลำดับจากระยะห่างว่า หัวที่จะอ่านกับปัจจุบัน จากน้อยไปมาก
ทำให้ข้อมูลที่อยู่ไกลจากตำแหน่งปัจจุบัน ถูกอ่านทีหลังเลย starvation
ก็ → เปลี่ยน algorithm

- 1.2. อุปกรณ์ภายนอกแบบใดไม่เหมาะกับการใช้งานแบบ Interrupt พร้อมอธิบายเหตุผล (5 คะแนน)

device ที่ transfer data เร็ว
เพราะ เสียเวลา กับ การทำ interrupt

- 1.3. การส่งงานจาก Processor ไปยัง I/O Controller สามารถทำได้ 2 รูปแบบคือ แบบแรกใช้คำสั่ง Generic I/O ที่ทำงานร่วมกับ Memory-mapped I/O และแบบที่สองคือใช้คำสั่ง Special I/O ที่ส่งตรงไปยัง I/O Controller แต่ละตัว ทั้งสองแบบมีข้อดีข้อเสียแตกต่างกันอย่างไร พร้อมยกตัวอย่างสถานการณ์ที่ OS ควรใช้ทั้ง 2 วิธีส่งการ I/O Controller ตัวเดียว (10 คะแนน)

แบบ Memory-mapped I/O

↳ device register ถูก mapped ลง memory space

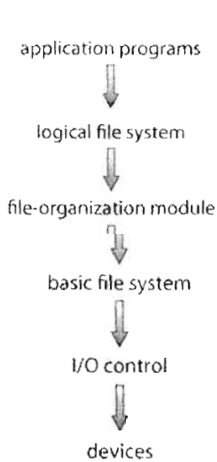
เพื่อใช้คำสั่ง store/load ปกติ

แบบ special I/O command

↳ ใช้คำสั่งพิเศษ read/write ไปที่ device register โดยตรง

สถานการณ์ที่ใช้ ทั้ง 2 → ไม่รู้

2. จากเนื้อหาที่เกี่ยวกับระบบไฟล์ (File System) จงตอบคำถามต่อไปนี้ (15 คะแนน)



2.1. จากแผนภาพแสดงความสัมพันธ์ของแต่ละโมดูลที่สำคัญ ที่จะทำให้แอปพลิเคชันต่างๆ (Application Programs) สามารถเข้าถึงข้อมูลที่อยู่ในฮาร์ดดิสก์ได้ (Harddisk Devices) จงอธิบายความรู้ที่มีต่อแผนภาพนี้ให้ได้มากที่สุด โดยเน้นการอธิบายในโมดูลที่เกี่ยวกับการทำงานในระบบปฏิบัติการ สำหรับโมดูลอื่นๆ อธิบายพอสังเขป (12 คะแนน)

device ถูกควบคุมโดย I/O control
 basic file system จะเป็นคนแปลรข้อมูลของ
 แต่ละ device ให้เป็นรูปแบบเดียวกัน
 และจัดการเปลี่ยน physical ให้เป็น logical
 เพื่อให้ app เข้าไปใช้งาน

2.2. เทคโนโลยี Redundant Array of Inexpensive Disks มีความสามารถที่โดดเด่นอะไรบ้าง เมื่อเทียบกับการใช้ฮาร์ดดิสแบบธรรมดา (3 คะแนน)

เพิ่ม reliability เพิ่ม performance

3. จากความรู้ที่ได้จากการนำเสนอผลงานของ Assignment ที่ 2 จงตอบคำถามต่อไปนี้ (10 คะแนน)

3.1. จงวิจารณ์ความแตกต่างของการใช้ฟังก์ชัน malloc กับ vmalloc (4 คะแนน)

3.2. Completely Fair Scheduler เป็นตารางการจัดงานที่ลินุกซ์ Ubuntu ใช้อยู่ในเวอร์ชันปัจจุบัน ซึ่งสามารถแก้ปัญหา Starvation ได้อย่างไร (3 คะแนน)

3.3. คำว่า traps กับ interrupt เหมือนหรือแตกต่างกันอย่างไร (3 คะแนน)

ไม่ได้ทำ assign 2 คำ

